

1 Experiments

1.1 Datasets

We evaluate NeuroGS-Codec on volumetric microscopy datasets containing neurite-like filamentary structures and sparse background. All volumes are stored as 3D tensors $V \in [0, 1]^{Z \times Y \times X}$ after intensity normalization. In our main results, we use [DATASET NAME(S)] with imaging modality [e.g., confocal / light-sheet / two-photon], voxel spacing $[(\Delta x, \Delta y, \Delta z)]$, and typical volume size $[Z \times Y \times X]$. We provide additional results on [SECOND DATASET / ADDITIONAL SAMPLES] to test generalization across acquisitions.

Preprocessing. Each volume is normalized using min-max normalization, $V \leftarrow (V - \min(V)) / (\max(V) - \min(V) + \epsilon)$, with $\epsilon = 10^{-8}$. To bias sampling and reconstruction toward neurite structures, we compute a neurite saliency map $M \in [0, 1]^{Z \times Y \times X}$ using a Difference-of-Gaussians (DoG) and gradient magnitude proxy (details in Sec. [METHOD SECTION REF]). [IF YOU HAVE TRAIN/VAL SPLITS: describe how you split volumes or patches.]

1.2 Baselines

We compare NeuroGS-Codec against classical and neural baselines representing common trade-offs in volumetric compression.

Lossless classical codecs. We include TIFF-LZW, TIFF-Deflate, HDF5+gzip, and Zarr+Blosc-zstd as standard archival baselines.

Lossy classical codecs. We include JPEG2000 (slice-wise) and H.265/HEVC (treating z -slices as video frames). For 2D codecs, we compress each axial slice independently and reconstruct the full volume by stacking slices.

Neural implicit baselines. We include coordinate-based implicit neural representation (INR) baselines: (i) SIREN-3D, and (ii) PE-MLP (Fourier features). We ensure comparable optimization budgets and compute bitrate via 8-bit weight quantization + gzip compression.

Implementation notes for fairness. All methods operate on the same normalized input volumes and are evaluated on the same metrics. Where applicable, we report both *compressed size* (true bytes on disk) and *estimated rate* (e.g., entropy surrogate), clearly labeled in tables.

1.3 Evaluation Metrics

We evaluate rate-distortion performance and geometric fidelity.

Rate. We report bits-per-voxel (bpv) defined as $\text{bpv} = \frac{8 \cdot \text{file_size_bytes}}{ZYX}$. For neural methods, we compute rate as serialized parameter bytes (8-bit quantized + gzip compressed).

Distortion. We report (i) PSNR, (ii) 3D SSIM, and (iii) slice-wise SSIM averaged across z . Additionally, to emphasize morphology-critical regions, we report an edge-focused error using a high-gradient mask E (top 10% gradients) and a flat-region mask F :

$$\text{MAE}_{\text{edge}} = \frac{1}{|E|} \sum_{(z,y,x) \in E} \left| \hat{V}_{z,y,x} - V_{z,y,x} \right|, \quad (1)$$

$$\text{MAE}_{\text{flat}} = \frac{1}{|F|} \sum_{(z,y,x) \in F} \left| \hat{V}_{z,y,x} - V_{z,y,x} \right|. \quad (2)$$

Speed and memory. We report encoding time, decoding time, and peak GPU memory. Decoding time is measured as the time to reconstruct the full volume at native resolution.

[OPTIONAL: If you compute morphological metrics (skeleton F1 / connectivity), add here.]

1.4 Training and Implementation Details

NeuroGS-Codec represents the volume as a sparse mixture of anisotropic 3D Gaussians with learnable means μ_i , scales s_i , orientations R_i (quaternion parameterization), and amplitudes a_i , plus a global bias b . We initialize Gaussians by sampling μ_i from the neurite map M and set initial amplitude a_i from voxel intensities. We train using Adam with learning rate schedule [LR schedule], for [number] iterations and batch size [B].

Sampling. Each iteration samples a minibatch of coordinates consisting of uniform samples and neurite-biased samples proportional to M . We weight reconstruction errors using $w(\mathbf{x}) = 1 + \kappa M(\mathbf{x})$.

Densification and pruning. We apply gradient-driven densification by splitting Gaussians with high position gradients and pruning primitives with low amplitude or excessive scale. We densify every [N] iterations from iteration [start] to [end] with maximum number of Gaussians [N_{max}].

Quantization and rate surrogate. We use straight-through rounding for quantized parameters and a Laplace entropy model to estimate rate in bits.

Figure 1: **Rate-distortion curves.** NeuroGS-Codec vs baselines on [DATASET]. Left: PSNR vs bpv. Right: 3D SSIM vs bpv. [Add notes: whether rate is true compressed bytes or estimated entropy.]

Figure 2: **Qualitative comparisons (MIP).** Ground truth, reconstruction, and absolute error MIP for [DATASET/SAMPLE] at [bpv]. [Optionally include multiple methods in rows.]

1.5 Main Results: Rate-Distortion and Fidelity

We present rate-distortion curves comparing NeuroGS-Codec to baselines across multiple bpv operating points. Figure 1 shows RD curves for PSNR and 3D SSIM. NeuroGS-Codec achieves [state your main claim: e.g., higher PSNR at same bpv / lower bpv at same SSIM] relative to [best baseline].

Qualitative comparisons. Figure 2 shows maximum intensity projections (MIPs) of ground truth, reconstructions, and absolute error maps. NeuroGS-Codec reconstructs thin neurite branches with high fidelity, while [baseline] exhibits [blurring / ringing / discontinuities].

Edge vs flat regions. To assess geometry-critical regions, we compute error separately on high-gradient edges and flat background areas. Figure 3 visualizes the gradient-based masks and the corresponding error maps. NeuroGS-Codec reduces edge-region errors by [X%] compared to [baseline], indicating improved boundary and filament preservation.

Quantitative summary. Table 1 reports the best-performing operating points at matched bpv for all methods. NeuroGS-Codec consistently improves [PSNR / SSIM / edge-MAE] while maintaining [decode time advantages / compact rate].

2 Ablation Studies

We perform ablations to isolate the contribution of each component in NeuroGS-Codec. Unless otherwise stated, all ablations use the same training iterations, sampling protocol, and evaluation settings as the full model.

2.1 Ablation Setup

We evaluate ablations at a matched operating point of [bpv] or matched number of primitives $N = [N]$. We report PSNR, 3D SSIM, and edge-

Figure 3: **Edge-focused analysis.** Gradient magnitude MIP, edge mask (top [10%]), flat mask, absolute error MIP, and error maps restricted to edge/flat regions.

Table 1: **Main quantitative results on [DATASET].** Report bpv (or bpp), PSNR, 3D SSIM, mean slice SSIM, MAE_{edge} , MAE_{flat} , decoding time, and model size.

Method	Rate (bpv)	PSNR $\uparrow$	SSIM _{3D} $\uparrow$	SSIM _{slice} $\uparrow$	MAE _{edge} $\downarrow$	MAE _{flat} $\downarrow$	Decode (s) $\downarrow$	Size (MB) $\downarrow$
TIFF-LZW	—	∞	1.0	—	—	—	—	—
JPEG2000 (slice)	—	—	—	—	—	—	—	—
H.265 (video-slices)	—	—	—	—	—	—	—	—
SIREN-3D	—	—	—	—	—	—	—	—
PE-MLP	—	—	—	—	—	—	—	—
NeuroGS-Codec (ours)	—	—	—	—	—	—	—	—

focused errors to quantify geometric fidelity.

2.2 Effect of Gaussian Parameterization

We compare anisotropic oriented Gaussians (full model) against isotropic Gaussians and/or Gaussians without learnable rotation:

- **Isotropic (no anisotropy):** $s_{i,x} = s_{i,y} = s_{i,z}$.
- **No rotation:** fix $R_i = I$.

This tests whether orientation-aware covariance is necessary for preserving neurite geometry and branching continuity.

2.3 Effect of Densification and Pruning

We remove densification and train with a fixed number of primitives ($N = N_0$), and compare against gradient-driven densification. We additionally ablate pruning thresholds (**min amplitude**, **max scale**) to understand rate allocation and artifact formation.

2.4 Loss and Regularization Ablations

We ablate each loss/regularization term:

- **No overlap penalty** ($\beta_{\text{overlap}} = 0$): tests whether separating Gaussians reduces redundancy and improves geometry.

Table 2: **Ablations on [DATASET/SAMPLE]** at matched [bpv or N].

Variant	Rate (bpv)	PSNR $\uparrow$	SSIM $_{3D}$ $\uparrow$	MAE $_{\text{edge}}$ $\downarrow$	Decode (s) $\downarrow$	#Gaussians N
Full (ours)	–	–	–	–	–	–
– overlap ($\beta_{\text{overlap}} = 0$)	–	–	–	–	–	–
– edge ($\beta_{\text{edge}} = 0$)	–	–	–	–	–	–
– SSIM ($\beta_{\text{ssim}} = 0$)	–	–	–	–	–	–
– TV ($\beta_{\text{tv}} = 0$)	–	–	–	–	–	–
– topology ($\alpha = 0$)	–	–	–	–	–	–
– conditioning ($\beta_{\text{smooth}} = 0$)	–	–	–	–	–	–
Isotropic Gaussians	–	–	–	–	–	–
No rotation ($R_i = I$)	–	–	–	–	–	–
Fixed N (no densify)	–	–	–	–	–	–

- **No edge-aware loss** ($\beta_{\text{edge}} = 0$): tests boundary preservation.
- **No SSIM term** ($\beta_{\text{ssim}} = 0$): tests structural perceptual similarity.
- **No TV term** ($\beta_{\text{tv}} = 0$): tests piecewise smoothness of reconstructions.
- **No topology proxy** ($\alpha = 0$): tests whether patch topology regularization helps preserve branching structures.
- **No parameter conditioning** ($\beta_{\text{smooth}} = 0$): removes scale/anisotropy constraints on Gaussian covariances.

Discussion. We expect β_{edge} and α to most strongly impact thin filament recovery, while β_{overlap} improves rate allocation and reduces redundant primitives. Parameter conditioning stabilizes optimization and prevents degenerate Gaussians.

2.5 Ablation Results

Table 2 summarizes quantitative results. Removing overlap regularization increases redundancy and slightly degrades [edge MAE / SSIM], while removing edge-aware loss increases boundary errors ([X%]). Disabling densification significantly reduces quality at thin branches, confirming the importance of adaptive primitive placement.

2.6 Qualitative Ablations

Figure 4 visualizes typical failure modes. Without edge-aware loss, reconstructions exhibit blurred neurite boundaries. Without overlap regularization, redundant Gaussians concentrate in high-intensity regions and pro-

Figure 4: **Qualitative ablations.** MIPs of representative reconstructions (full vs ablated variants) and corresponding absolute error maps.

Table 3: **Complexity comparison.** Encode/decode time and peak memory on [\[GPU/CPU details\]](#).

Method	Encode time (s)↓	Decode time (s)↓	Peak VRAM (GB)↓	Peak RAM (GB)↓
JPEG2000	–	–	–	–
H.265	–	–	–	–
SIREN-3D	–	–	–	–
PE-MLP	–	–	–	–
NeuroGS-Codec (ours)	–	–	–	–

duce [\[artifacts / oversmoothing\]](#). Without densification, thin branches are frequently missed.

2.7 Complexity Analysis

We report encoding/decoding time and memory footprint to quantify practical usability. NeuroGS-Codec provides [\[fast decoding / interactive reconstruction\]](#) compared to INR baselines that require evaluating an MLP at every voxel. Table 3 summarizes runtime and memory.